Informe de Arquitectura e Infraestructura Técnica - ClearBid NJ

Fase de Presentación Formativa (Nota 3 - Capstone Project)

1. Resumen Ejecutivo

El presente documento detalla la arquitectura de software, infraestructura en la nube y estrategias de integración continua implementadas para el proyecto ClearBid NJ. El objetivo principal de esta fase ha sido establecer un entorno de producción robusto, escalable y de alta disponibilidad, resolviendo bloqueos críticos de comunicación entre servicios y asegurando un flujo de trabajo unificado para el equipo de desarrollo.

2. Arquitectura Frontend (Vercel + React/Vite)

- **Decisión Arquitectónica:** Se optó por separar el entorno visual del backend, creando un repositorio dedicado exclusivamente para el Frontend. Esto permite una integración nativa y optimizada con **Vercel**.
- **Problemas Resueltos:**
    - **Enrutamiento en Single Page Applications (SPA):** Al recargar páginas en producción, el servidor arrojaba errores "404 Not Found". Se solucionó implementando reglas de reescritura globales mediante un archivo `vercel.json` y migrando el sistema de rutas a un enrutador basado en Hash (`HashRouter`), garantizando la persistencia de la navegación.
- **Valor Agregado:** Vercel proporciona una CDN (Content Delivery Network) global que minimiza la latencia, ofreciendo una experiencia de usuario rápida. Además, se configuró el sistema para detectar automáticamente el idioma del navegador del usuario (`navigator.language`), mejorando la accesibilidad internacional.

3. Arquitectura Backend y Base de Datos (Render + Supabase + FastAPI)

- **Decisión Arquitectónica:** El núcleo lógico del sistema fue desarrollado en **Python 3.12** utilizando el framework **FastAPI** por su alto rendimiento asíncrono. Este servicio se desplegó en **Render**.
- **Gestión de Datos:** La persistencia de datos se maneja a través de **Supabase** (PostgreSQL en la nube). La base de datos actual ha sido poblada de manera preliminar estructurando información extraída de los documentos del condado de Mercer y enlaces provistos por el cliente, estableciendo el esqueleto funcional del proyecto.

- **Seguridad y Conectividad:** Se configuraron estrictas políticas de **CORS (Cross-Origin Resource Sharing)** en el backend para permitir únicamente peticiones provenientes del dominio oficial de Vercel y de los entornos de desarrollo local (`localhost:5173`). Todas las credenciales sensibles están protegidas mediante variables de entorno (`.env`).

4. Estrategia de Alta Disponibilidad (Integración de UptimeRobot)

- **Desafío Técnico:** Los servicios alojados en la capa gratuita de Render (Free Tier) entran en un estado de hibernación tras 15 minutos sin recibir tráfico. Esto provocaba un "Cold Start" (arranque en frío), causando que el primer usuario que intentara iniciar sesión o registrarse experimentara tiempos de carga de hasta 60 segundos o errores de red (`Network Error`).
- **Solución Implementada:** Se integró el servicio de monitoreo **UptimeRobot**.
- **Mecanismo:** Se configuró un "bot" que envía peticiones HTTP (pings) al endpoint de estado del backend cada 5 minutos.
- **Impacto:** El servidor se mantiene en estado "Awake" (despierto) de forma ininterrumpida (24/7), reduciendo el tiempo de respuesta inicial de 60 segundos a milisegundos y garantizando una experiencia de usuario fluida desde el primer clic.

5. Sistema de Autenticación y Mensajería Segura (API de Brevo)

- **Desafío Técnico:** El sistema de validación de usuarios requería el envío de correos con códigos de un solo uso (OTP/2FA). Inicialmente, la implementación mediante el protocolo estándar SMTP (usando servidores como Gmail) funcionaba en local, pero era bloqueada sistemáticamente al ser desplegada en la nube por restricciones de puertos y políticas antispam.
- **Solución Implementada:** Se sustituyó completamente el protocolo SMTP por integraciones de API HTTPS utilizando el proveedor **Brevo**.
- **Ajustes de Negocio:** Basado en el feedback de los stakeholders, se extendió el tiempo de validez del código de doble factor (2FA) de 5 minutos a **15 minutos** para mayor comodidad del usuario final.
- **Impacto:** Se logró una tasa de entregabilidad de correos del 100%, evitando bloqueos de red y garantizando que los usuarios puedan completar sus procesos de registro y acceso sin interrupciones.

6. Control de Versiones y Estandarización del Equipo (GitHub)

- **Estrategia de Ramas:** Se implementó una sincronización unidireccional desde la rama principal (`main`) hacia las ramas de especialidad (`Backend`, `DevOps`, `database`).

- **Protección de Activos Académicos:** Mediante la ejecución controlada de `Pull Requests` e inyección de código directa, se logró unificar el código fuente de producción en todas las ramas sin sobreescribir ni eliminar los documentos de investigación y entregables académicos que el equipo ya había desarrollado en cada rama.
- **Entorno Local:** Se documentó y estandarizó el uso estricto de Entornos Virtuales (`venv`) con Python 3.12, asegurando que cualquier miembro del equipo pueda levantar el proyecto completo en su máquina sin generar conflictos de dependencias con otros proyectos.